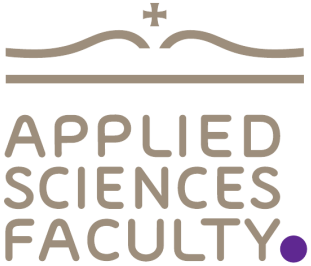
C++ — лекція 19. Потоки вводу-виводу





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

```
char a;
cout << "Enter a: ";
cin  >> a;
```

Об'єкти `cout` і `cin` — це екземпляри класів `ostream` і `istream`, відповідно, які, у свою чергу, є інстанціюванням `basic_ostream<char, char_traits<char> >` та `basic_istream<char, char_traits<char> >` — шаблонів вводу-виводу "чого-не-будь" для виводу символів, char.

По замовчуванню створюються наступні об'єкти:

- cout — вивід на екран, в консоль.
- cin — ввід з клавіатури.
- cerr — небуферизований вивід повідомлень про помилку.
- clog — буферизований вивід повідомлень про помилки, спеціальні ситуації та етапи роботи програми.

Буферизація

- Запис на диск чи виведення на екран — процес повільний.
- Щоб комп'ютер використовував свої ресурси ефективніше, використовується **буферизація** — дані, призначені для запису, накопичуються в спеціальній області пам'яті — буфері, потім виводяться одним пакетом.
- Ілюстрація, чому буферизація прискорює вивід. Запис на диск здійснюється так званими секторами, пакетами розміром 512 байт[1]. Щоб його записати, диск має перевести головки у правильне положення, дочекатися поки відповідний сектор пропливе під головкою, провести запис. Ганяти так головки для кожного сектора (який потрібно записати, навіть якщо змінюється лише один байт) окремо — неефективно! Звичайно, в сучасних системах все багато складніше — є багато рівнів буферизації. Однак, загальний принцип залишається тим же.
- Є можливість примусово спорожнити буфери — ініціювати операцію вводу-виводу. Один із способів — виведення `endl`.

[1] Сучасні жорсткі диски потроху переходять на використання секторів розміром 4Кб.

cout

На прикладі буферизації для cout розглянемо кілька способів роботи із об'єктами з iostream.

- Очистити буфер — примусити здійснити операцію виводу, навіть якщо буфер ще не заповнений, можна за допомогою виведення об'єкту endl. Для цього викликається:
- спеціальна функція-метод flush():

```
cout.flush();
```

- інший спосіб його викликати:

```
cout << flush;
```

І endl і flush — це так звані **маніпулятори**, сутності, виведення чи введення яких, змінює стан потоку.

put() і write()

Низькорівневими методами ostream є функції `put()` і `write()`. Перша із них виводить на екран один символ і повертає посилання на той же об'єкт cout[2]:

```
ostream &basic_ostream::put(char T);
```

Тому, її можна використовувати не тільки так:

```
cout.put('A');
cout.put('B');
cout.put('C');
```

Але і так:

```
cout.puts('A').puts('B').puts('C');
```

що еквівалентно:

```
( ( cout.put('A') ).put('B') ).put('C');
```

[2]

write(), у свою чергу, виводить вказану кількість символів:

```
ostream& write (const char* s, streamsize n);
```

puts() і write(), в основному, призначені для використання самим класом — для користувачів вони занадто низькорівневі.

operator< <

На практиці для виводу ми пишемо щось таке:

```
cout << "Something";
cout << " Once more." << endl;
```

Справді, для ostream та istream перевизначені оператори `operator<<` та `operator>>`, відповідно, для всіх фундаментальних типів.

Тобто, коли написати:

```
int a=33;
cout << a;
```

викликається оператор:

```
ostream& operator<< (int val);
```

Якщо ж писати:

```
bool b=false;
cout << b;
```

викликається:

```
ostream& operator<< (bool val);
```

Можна писати також свої перевизначення.

Наприклад, для розглянутого раніше типу OurVector, можна написати:

```
ostream &operator<<(ostream &out,const OurVector& o)
{
    out << "(" << o.get_x() << "," << o.get_y() << ")";
    return out;
}
```

Тоді:

```
OurVector A(2,3);
cout << "A = " << A << endl;
```

виведе:

```
A = (2,3)
```

Форматування

Нехай у нас є наступна програма:

```
int a=567777, b=33234, c=33333;
cout << a << b << c << endl;
```

На екран, на жаль, вона виведе щось таке:

```
5677773323433333
```

І це — далеко не завжди те, що потрібно.

Просте рішення:

```
int a=567777, b=33234, c=33333;
cout << a << " " << b << " " << c << endl;
```

На екран вона виведе щось таке:

```
567777 33234 33333
```

Вже краще.

Ускладнюємо задачу:

```
int a1=567777, b1=33234, c1=33333;
int a2=99,     b2=61,    c2=11;

cout << a1 << " " << b1 << " " << c1 << endl;
cout << a2 << " " << b2 << " " << c2 << endl;
```

На екран буде виведено таке:

```
567777 33234 33333
99 61 11
```

Все рівно — зле.

Функція width() та маніпулятор setw()

```
cout.width(15);
```

скаже, що наступний вивід займатиме 15 символів.

Якщо вивід займатиме менше, решту буде заповнено пробілами. **УВАГА:** якщо більше — виведуться всі символи, C++ вважає, що краще знехтувати форматуванням, ніж загубити частину виводу[3].

Якщо включити:

```
#include <iomanip>
```

можна буде також робити так:

```
cout << setw(15) << 1234;
```

На жаль, як width(), так і setw() діють лише для одного наступного виводу.

[3] Деякі мови вважають по іншому!

```
cout << setw(15) << a1 << setw(15) << b1 << setw(15) << c1 <
cout << setw(15) << a2 << setw(15) << b2 << setw(15) << c2 <
```

На екран, на жаль, вона виведе щось таке:

```
 567777          33234          33333
     99             61             11
```

Вже краще.

Маніпулятори — файл <iomanip>

`setw()` — ще один приклад **маніпулятора**, такого, який приймає аргументи.
Виведення маніпулятора в потік змінює режим виведення.
Для маніпуляторів існують відповідні методи класів вводу-виводу.
Виведення в об'єкт маніпулятора якраз приводить до виклику цих методів.

УВАГА! Ефект більшості маніпуляторів, на відміну від setw(), триватиме, поки не буде зміненим явно!

Маніпулятори

- setfill('0') — встановити заповнювач, (замість пробілу).
- boolalpha / noboolalpha — виводити булівські типи як "true"/"false", чи як "0"/"1".
- showpos / noshowpos — дозволити чи заборонити явне виведення "+" для додатних чисел.
- dec / hex / oct — виводити числа у десятковій/шістнадцятковій/вісьмірковій системах.
- showbase/noshowbase — показувати/не показувати основу системи числення.
- showpoint/noshowpoint — ввімкнути/вимкнути примусове виведення десяткової крапки.
- fixed/scientific/defaultfloat (C++11) — завжди використовувати форму запису із фіксованою крапкою/наукову форму запису/по замовчуванню.
- setprecision(7) — встановити кількість десяткових знаків для виводу.
- left/right/internal — вирівнювати по лівому/правому краю, або знак ліворуч, число – праворуч, символ-заповнювач в середині.
- unitbuf/nounitbuf — ввімкнути/вимкнути спорожнення буферів після кожного виводу.
- Детальніше див. https://en.cppreference.com/w/cpp/header/iomanip

Приклад setfill

```
cout << setfill('0');
cout << setw(15) << 1234 << endl;
cout << setw(15) << 7890 << endl;
cout << setfill(' ');
cout << setw(15) << 7890 << endl;
```

На екран буде виведено:

```
000000000001234
000000000007890
           7890
```

std::quoted

- Маніпулятор std::quoted дозволяє читати та записувати в потоки символи, захищені escape-послідовностями:

```
    std::stringstream ss;
std::string in = "String with spaces, and embedded \"quotes\" too";
std::string out;
ss << quoted(in);
ss >> quoted(out);
```

- Дозволяють обирати символ-роздільник, який потрібно захищати і символ escape-послідовності.
- Детальніше див. `https://en.cppreference.com/w/cpp/io/manip/quoted`.

Приклад boolalpha

```
bool t=true, f=false;
cout << t << "," << f << endl;
cout << boolalpha;
cout << t << "," << f << endl;
cout << noboolalpha;
cout << t << "," << f << endl;
```

На екран буде виведено:

```
1,0
true,false
1,0
```

Введення та об'єкт cin

Цей об'єкт, типу `istream`, в значній мірі схожий на cout, тільки відповідає за прийом даних, введених з клавіатури.

Для нього перевизначено оператор `operator>>` , тому для коду:

```
int a;
char b;
bool c;

cin >> a >> b >> c;
```

будуть викликатися відповідні перевантажені `operator>>`,

```
ostream& operator>> (int);
ostream& operator>> (char);
ostream& operator>> (bool);
```

Для istream теж перевантажено `operator>>` для всіх фундаментальних типів.

Символи-роздільники

operator>> завершує читання, зустрівши символ-роздільник. По замовчуванню це пробіли, табуляції, символи кінця рядка[4]. Тому, наприклад, прочитати стрічку з пробілами так не вдасться:

```
char str1 [100];

cin >> str1;
cout << str1 << endl;
```

покаже:

```
fff bbb csas          <=== введено
fff                   <=== виведено
```

[4] Формально — все, для чого функція `int isspace ( int c );` з файлу <cctype> повертає true.

Сімейство функцій get()

Для вирішення цієї проблеми можна скористатися однією із функцій get():

- `istream& get (char& c)` — читає одну літеру.
- `istream& get (char* s, streamsize n)` — читає стрічку, зберігаючи її в s, поки не зустрінеться символ кінця рядка (користувач не натисне Enter), але не більше ніж n символів, включаючи нульовий символ `'\0'`, який автоматично додає в кінці. Символ кінця рядка **залишається в потоці!**. Якщо символ кінця рядка не трапився, коли кількість записаних символів досягла n, встановлює failbit (див. далі).
- `istream& get (char* s, streamsize n, char delim)` — те ж, що і попередня функція, але поки не зустрінеться символ delim.

getline()

Ще одна альтернатива — функція getline():

```
istream& getline (char* s, streamsize n );
istream& getline (char* s, streamsize n, char delim );
```

Вона теж читає не більше, ніж n символів, (включаючи '\0', який додається в кінці автоматично), поки не зустрінеться символ кінця рядка або delim, відповідно. На відміну від get(), символ кінця рядка **забирається** з потоку!

```
cout << "String 1: ";
cin.get(str1,99);
cout << str1 << endl;
cout << "String 2: ";
cin.get(str1,99,'e');
cout << str1 << endl;
```

```
String 1: aaa bbb ccc
aaa bbb ccc
String 2: abcdefg
                          <=====   '\n'
abcd
```

Зауважте — якщо б наступна get() не використовувала інший роздільник, не прочиталося б нічого! '\n' залишився в потоці, і наступні звернення зразу б завершували читання.

Альтернативний спосіб:

```
cout << "String 1: ";
cin.getline(str1,99);
cout << str1 << endl;
cout << "String 2: ";
cin.getline(str1,99,'e');
cout << str1 << endl;
```

Результат:

```
String 1: aaa bbb ccc
aaa bbb ccc
String 2: abcdefg
abcd
```

std::getline

- Більш високорівнева альтернатива, оголошена в <string>:

```
istream& getline (istream& is, string& str, char delim);
istream& getline (istream& is, string& str);
```

- Її перевага в тому, що немає потреби турбуватися про розмір буфера — стрічка-аргумент сама збільшуватиметься за потреби.
- Подробиці див. `http://en.cppreference.com/w/cpp/string/basic_string/getline`

ignore()

Якщо після get() чи getline() прочитано не всі символи, але вони нам не потрібні, їх можна просто проігнорувати:

```
istream& ignore (streamsize n = 1, int delim = EOF);
```

peek()

Іноді дуже хочеться спершу побачити, що є в потоці, і лише потім — по-справжньому прочитати. Приклад: програма, яка вводить числа, поки не користувач не введе 'x'.

Для цього служить функція peek():

```
int sum=0;
while(true){
    cout << "Enter next number or 'x' to exit : ";
    char ck=cin.peek();
    if(ck=='x' || ck=='X' ){
            cout << "Sum = " << sum << endl;
            break;
    }
    int a;
    cin >> a;
    sum += a;
}
```

Альтернативний підхід — putback()

Можна, також, замість підглядати, спочатку дістати символ з потоку, а потім, якщо не наш, повернути назад:

```
while(true){
    cout << "Enter next number or 'x' to exit : ";
    char ck;
    cin >> ck;
    if (ck=='x' || ck=='X' ){
        cout << "Sum = " << sum << endl;
        break;
    }else{
        cin.putback(ck);
    }
    int a;
    cin >> a;
    sum += a;
}
```

Ще одна альтернатива — unget()

Функція unget() намагається зробити так, ніби останній прочитаний символ не було прочитано.

```
istream& unget();
```

Спрацьовує не завжди — залежить від реалізації.

Низькорівні функції читання — read і readsome

Аналогічно до методу write в ostream, в istream існує метод read:

```
istream& read (char* s, streamsize n);
```

Функція ця зовсім низькорівнева — просто читає вказану кількість символів. Нульовий символ в кінці не додає!

Крім того, існує функція readsome, яка читає не більше n символі, але лише ті, що наявні в буфері — не чекає, поки набереться рівно n символів.

```
streamsize readsome (char* s, streamsize n);
```

gcount()

Ця функція повертає кількість символів, прочитаних останньою низькорівневою функцією читання:

streamsize gcount() **const**;

Низькорівневими є get, getline, ignore, peek, read, readsome, putback і unget. Після peek, putback і unget ця функція поверне 0.

Маніпулятори для cin

Клас istream, екземпляром якого є cin, теж має свої маніпулятори. Поміж них:

- boolalpha / noboolalpha — читати булівські змінні як "true"/"false" чи як "0"/"1".
- skipws/noskipws — перш, ніж читати, пропускати/не пропускати пробіли та інші символи-роздільники.
- dec / hex / oct — читати введене, як десяткове/шістнадцяткове/вісьміркове число.
- **setw(n)** — встановлює, що наступне читання в масив char (C-стрічку) прочитає максимум n-1 символ.
- Детальніше див. `www.cplusplus.com/reference/library/manipulators/`

Використання їх трішки менш очевидне — не їх виводиться потік, а, умовно, здійснюється читання "в них":

```
cin >> skipws;
```

Насправді, в обох випадках, це всього лиш виклик перевантаженого відповідним чином оператора < < чи > >, який, у свою чергу, змінює стан об'єкту.

```
int A,B;
cin >> dec;
cin >> A;

cin >> hex;
cin >> B;

cout << endl << A << endl << B << endl;
```

Якщо двічі ввести 100, першого разу його прочитає як десяткове сто, другий раз як шістнадцяткове, що відповідає десятковому 256:

```
100
100

100
256
```

Стан потоку

Стан потоку зберігається у спеціальних прапорцях[5]. Подивитися їх можна за допомогою відповідних методів:

- `good()` — можна виконувати наступну операцію вводу-виводу.
- `eof()` — було досягнуто кінець файлу.
- `fail()` — наступна операція не виконається.
- `bad()` — потік зіпсовано.

```
cout << "Enter int number : ";
int x;
cin >> x;
cout << endl;
cout << "good: " << cin.good() << endl;
cout << "fail: " << cin.fail() << endl;
cout << "bad: " << cin.bad() << endl;
```

Маніпулювати цими бітами можна за допомогою наступних методів:

```
iostate rdstate() const; // Прочитати
void setstate(iostate f); // Встановити
void clear(iostate f=goodbit);// Очистити
```

[5] Вони визначені в `ios_base::iostate`, і є окремими бітами в певному "бітовому полі".

Неявна перевірка стану потоку

Потік istream має оператор неявного приведення до `void*` і перевизначає `operator!()`. Перший із них повертає не нульове значення, тобто — true, якщо !fail(), другий — якщо fail(). Таким чином, читати із файлу, поки він не закінчиться, можна якось так:

```
int x;
while(mystream>>x)
{
    cout << x;
}
```

- Якщо get() чи getline() не змогли прочитати хоча б один символ, вони встановлюють failbit.
- `operator>>` встановлює failbit, якщо не зміг коректно прочитати значення того типу, у змінну якого відбувається читання. В цьому випадку змінна збереже вихідне значення. Потік не є пошкодженим, тому після clear() можна спробувати читати знову.

Виключення потоків

По замовчуванню виключення не генеруються — вважається, що збої вводу-виводу достатньо поширені, щоб оброблятися в основному потоці програми, а не за допомогою виключень[6]. Це не завжди зручно, тому їх обробку можна ввімкнути відповідними методами:

```
iostate exceptions() const;
void exceptions( iostate except );
```

Наприклад:

```
ios_base::iostate def = cin.exceptions();
// Зберігаємо конфігурацію виключень
cin.exceptions(ios_base::badbit | ios_base::failbit );
// Нижче генеруватимуться виключення для ситуацій fail() та bad()
  .....
cin.exceptions(def);
// Відновлюємо конфігурацію виключень, що була раніше
```

[6] Ну і, потоки з'явилися помітно раніше, ніж виключення.

Трішки про реалізацію потоків

- **Прапорці форматування**
- Детально реалізацію форматування, яким керують маніпулятори, ми розглядати не будемо, однак для цього теж використовуються певні бітові поля.
- Для збереження і відновлення стану потоку можна використати метод copyfmt().

```
basic_ios& copyfmt(const basic_ios& fmt);
```

- Вона копіює практично весь стан потоку, окрім буферу та його стану. Таким чином, можна переносити форматування між різними потоками.

Приклад використання copyfmt

- RAII обгортка, ка зберігає стан потоку (окрім width) та відновлює його при знищенні.

```
template <typename charT = char,
          typename traits = std::char_traits<charT>>
class ostream_state_restorer_t{
    using ostream_type = std::basic_ostream<charT, traits>;
    using ostream_saver_type = std::basic_ostringstream<charT, traits>;

    ostream_type &os;
    ostream_saver_type stream_state_store;
public:
    ostream_state_restorer_t(ostream_type& os_): os(os_) {
        stream_state_store.copyfmt(os);
    }
    ~ostream_state_restorer_t(){
        os.copyfmt(stream_state_store);
    }
};
```

- Приклад використання:

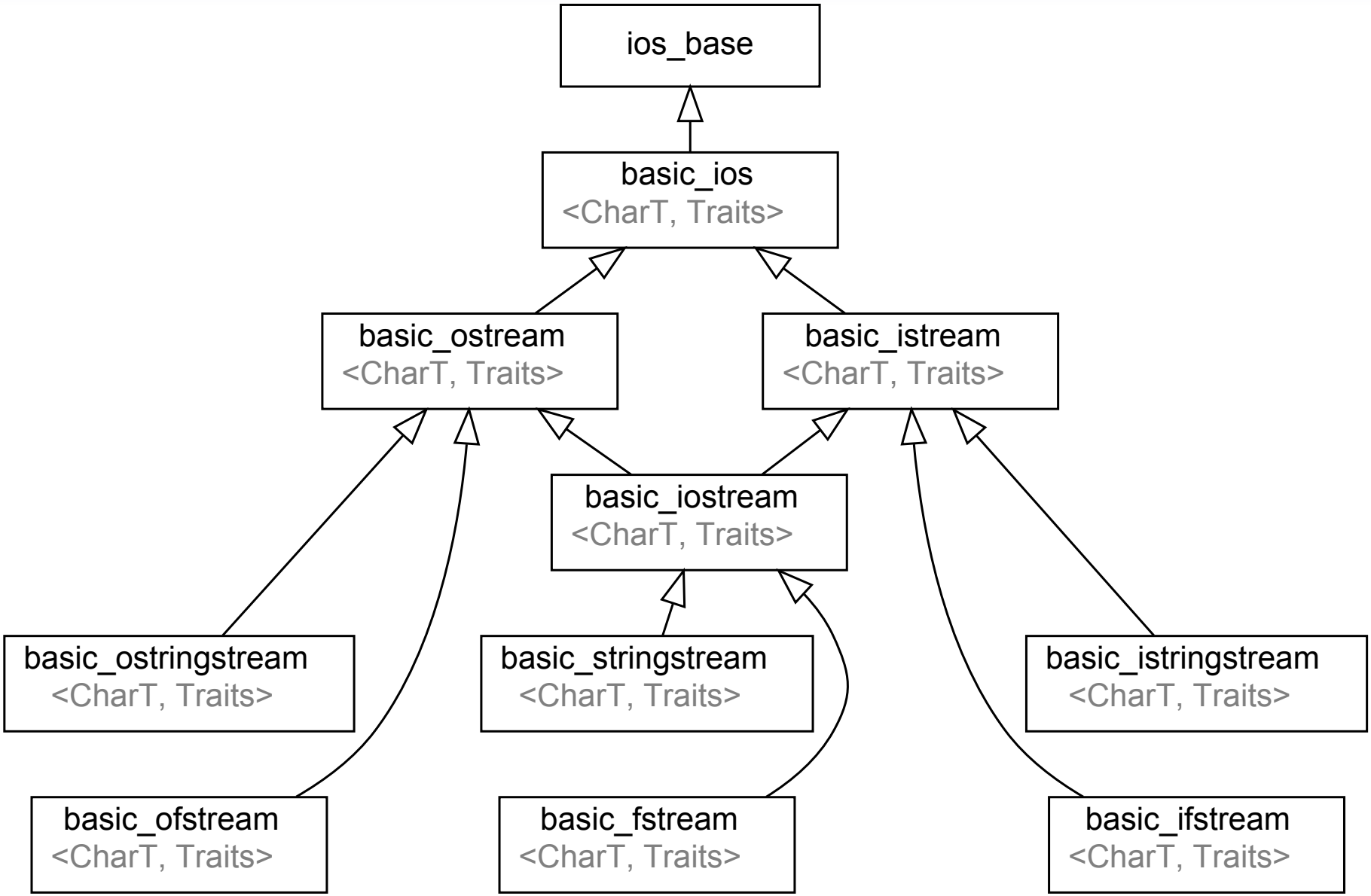
```
{
        ostream_state_restorer_t sr(cout);
        cout << uppercase << hex << setfill('0') << right;
            ..............................
}
```

- cout – об'єкт глобальний, змінювати його стан, особливо в функціях, не варто, це впливатиме на поведінку цілої програми. А так він відновлюватиметься автоматично.

- Об'єкти cout, cin, cerr, clog[7] — вже підготовлені.
- Абстракції файлів у C++, фактично, не існує. (Суб-стандартним методом є використання boost::filesystem, яку буде включено в наступний стандарт).
- Однак, можна створювати потоки, з тими ж самими можливостями, пов'язані із файлами.
- Для цього служать класи **ofstream** — файловий потік виводу, та **ifstream** — файловий потік вводу.
- Вони вміють все те, що і cout і cin, відповідно, але враховують деякі нюанси, важливі для роботи з файлами.
- Оголошені ці класи в <fstream>.

[7] Та їх "широкі" варіанти — wcout і т.д.

Ієрархія класів потоків[8]:



[8](c) cppreference.com.

- Спершу файл потрібно **відкрити** — пов'язати потік із реальним файлом на диску, або створити відповідний файл:

```
ofstream  file1 (" file . txt ");
```

- Відкритий таким чином файл буде обрізано — всі дані відкинуто, і запис почнеться з початку. Про інші режими відкриття файлів — див. далі.
- Щоб дані "остаточно" потрапили на диск, було встановлено відповідну метаінформацію – час модифікації, розмір, тощо, і з файлом можна було робити ще щось[9], файл слід **закрити**:

```
file1 . close ();
```

- Закривати файли після використання так само важливо, як і звільняти пам'ять!
- Деструктори файлових потоків закривають пов'язані із ними файли.
- Перевірити, чи файл відкрито, можна за допомогою методу:

```
bool is_open();
```

[9] Строго кажучи, це не обов'язкове, але пояснення всіх нюансів занадто сильно відволіче нас.

Записуємо у файл випадкові числа:

```
srand( time(0) );
ofstream file1 ("file.txt");
for(int i=0;i<15;++i)
{
    for(int j=0;j<3;++j)
    {
        file1 << setw(10) << rand();
    }
    file1 << endl;
}
file1.close();
```

Читання з файлу

- Читання дещо складніше — наприклад, слід пильнувати, чи не закінчилися дані.
- Однак, завдяки згаданим раніше операторам приведення до `void*` та `operator!()`, можна робити так:

```
ifstream file2 ("file.txt");
int r;
while( file2 >> r){
        .....
}
```

Цей код прочитає цілий файл.

Режими відкривання файлів

- Іноді є потреба розділити створення файлового потоку та пов'язування його з файлом — відкриття файлу.
- Для цього в класах потоків є аргумент конструкторів по замовчуванню та метод open():

```
void basic_ifstream::open(const char* filename,
        ios_base::openmode mode = ios_base::in);
explicit basic_ifstream::basic_ifstream (const char* filename,
        ios_base::openmode mode = ios_base::in);

void basic_ofstream::open (const char* filename,
        ios_base::openmode mode = ios_base::out);
explicit basic_ofstream::basic_ofstream (const char* filename,
        ios_base::openmode mode = ios_base::out);

void basic_fstream::open (const char* filename,
        ios_base::openmode mode = ios_base::in | ios_base::out);
explicit basic_fstream::basic_fstream (const char* filename,
        ios_base::openmode mode = ios_base::in | ios_base::out);
```

- Параметр **mode** задає режим відкриття. Режим комбінується за допомогою побітового "АБО" ("OR") між визначеними наперед константами з ios_base. Тобто, mode — бітове поле.

Константи ios_base

- **ios_base::in** — відкрити для читання (Вважається завжди встановленим для basic_ifstream).
- **ios_base::out** — відкрити для запису. (Вважається завжди встановленим для basic_ofstream)
- **ios_base::binary** — відкрити файл у двійковому режимі (на противагу текстовому).
- **ios_base::ate** — після відкриття файлу, вивід відбуватиметься в кінець.
- **ios_base::app** — ввід-вивід завжди відбуватиметься з кінця файлу.
- **ios_base::trunc** — вміст файлу перед початком роботи з ним відкидається.
- Не всі комбінації сумісні! Наприклад, немає сенсу писати app і trunc, детальніше про заборонені комбінації див. документацію.
- Різниця між ate і app наступна: ate переміщає файловий вказівник на кінець файлу при відкриванні, а app — за кожної операції запису, змушуючи, щоб запис відбувався лише в кінець. ate не запобігають trunc — цю опцію слід "вимикати" явно. app "вимикає" trunc.

В попередньому коді можна було писати:

```
//ofstream file1 ("file.txt");
fstream file1;
file1.open("file.txt", ios_base::out | ios_base::ate);

        ..................

//ifstream file2 ("file.txt");
fstream file2;
file2.open("file.txt", ios_base::in );
```

Файловий вказівник

- Читання з потоку та запис у нього відбувається з певного положення[10].
- Взагалі, потоки мають два види вказівників – для запису (із суфіксом p — put) і для читання (суфікс g — get), однак, через особливості реалізації всіх (більшості?) ОС, ці вказівники для файлових потоків завжди співпадають.
- Є можливість керувати файловим вказівником:

```
istream& seekg (streampos pos);
istream& seekg (streamoff  off , ios_base:: seekdir  way);
istream& seekp (streampos pos);
istream& seekp (streamoff  off , ios_base:: seekdir  way);
```

(та аналогічні функції для інших класів потоків — ostream, файлових, тощо).

- `streampos` — тип вказівника у файлі. Цілочисельний (інтегральний) тип даних, який, однак, може бути більшим, ніж int чи size_t на даній машині. `streamoff`, відповідний тип зі знаком, використовується, щоб казати зміщенню між двома положеннями у файлі.
- ios_base::seekdir може набувати одного із трьох значень, перемістити: beg — відносно початку, end — відносно кінця, cur — відносно поточного положення.
- Подивитися положення вказівника у файлі можна за допомогою методів `streampos tellp()` та `streampos tellg()` .

[10] На це натякає вже підбірка режимів відкриття файлів.

<spanstream> – C++23

- C++23 надав спосіб використовувати будь-які масиви як буфери потоків:

```
char input[] = "10 20 30";
std::ispanstream is{std::span<char>{input}};

char output[30]{}; // Ініціалізовано нулями
std::ospanstream std::os{span<char>{output}};
os << 10 << 20 << 30;

std::vector<char> data1(4*1024,'\0');
std::ospanstream out(data);

std::string data2{"10 20 30"};
std::ispanstream sss(data);
```

- Цей сучасний засіб, аналогічний застарілим з C++98 std::strstream (викинуто в C++26).

C++23 – інше

- std::ios_base::noreplace – прапорець для створення файлу лише якщо його ще не існує, уникаючи можливої гонитви, коли файл було створено іншим кодом, який виконувався паралельно, після перевірки, але до створення. Подробиці: Support exclusive mode for fstreams.

C++26

- Для файлових потоків C++26 надає доступ до нативного дескриптора, методом native_handle():

```
std :: ofstream  ofs{"data.txt"};
  .......
ofs.flush ();                          // Runtime --> OS buffers
flush(ofs.native_handle());  // Flush  OS buffers
```

- Запропоновано file_handle і mapped_file_handle – обгортки над відповідними низькорівневими засобами ОС. `https://wg21.link/p1883`.
- Запропоновано додати std::filesystem::path_view та std::filesystem::path_view_component[11]: `https://wg21.link/p1030`.

[11] Не може містити роздільника директорій.

C++ успадкував від C ряд засобів роботи із файлами, які часто все ще актуальні. Заглиблюватися у них ми не будемо, тому лише приклад:

```
FILE *pFile;
pFile = fopen ("the_file.txt","w");
if (pFile == 0){
    cout << "Error opening file!" << endl;
    return 1;
}

fputs ("Our string in file. La-la-la!\n",pFile);
fprintf (pFile,"a=%i, b=%i",11,22);
fclose (pFile);
```

Функції сімейства printf (та аналогічні їм функції читання, scanf), часто зручніші, ніж потоки, однак мають суттєві недоліки. Перший — їм потрібно явно вказувати тип аргументу, що є великим джерелом помилок. Другий — їх поведінку неможливо розширити для користувацьких типів (класів), та складно використовувати із шаблонами.

- Детальніше розібратися із "зоопарком" файлових класів та їх можливостей вам допоможе, наприклад, ось це посилання: `http://www.cplusplus.com/reference/iolibrary/`
- Важливий нюанс – стандарт вимагає робити flush чи seek переш, ніж змінювати напрямок – читання на запис і навпаки.
  - Цитата з https://stackoverflow.com/questions/23843587/fwrite-doesnt-work-directly-after-fread: "When a file is opened with update mode ('+' as the second or third character in the above list of mode argument values) [...] input shall not be directly followed by output without an intervening call to a file positioning function, unless the input operation encounters end- of-file."
  - Історія питання: "I Thought I Found a Bug...".